2023 IEEE International Conference on Web Services (ICWS) | 979-8-3503-0485-5/23/$31.00 ©2023 IEEE | DOI: 10.1109/ICWS60048.2023.00025

Functional and Structural Fusion based Web API Recommendations in Heterogeneous Networks

Xuanye Wang[1], #Meng Xi[1,2] Jianwei Yin[1,2],
[1]*School of Software Technology, Zhejiang University, Hangzhou, China*
[2]*School of Computer Science and Technology, Zhejiang University, Hangzhou, China*
corresponding author
Email: xuanyewang@zju.edu.cn, ximeng@zju.edu.cn, zjuyjw@cs.zju.edu.cn

***Abstract*—With the increasing development of cloud computing, a large number of innovative Mashup applications and Web APIs have emerged on the Internet. The expansion of technology and information presents a significant challenge to the discovery of Web APIs from multiple service ecosystems. Various Web API recommendation methods have been proposed in mashup creation, but most either assign equal weight to model factorization interactions or solely rely on requirements information for API recommendation. Unfortunately, these methods face several challenges, such as explicit and implicit dependencies among APIs, ambiguous API semantics, and the undervaluation of tail APIs. In this work, we propose a Functional and Structural Fusion Model (FSFM) based on Web API recommendation for Mashup creation. To address the above deficiencies, we first design the structural interaction component to encode the latent structural relationships in the heterogeneous network of Mashups and APIs and capture the topological structure signals between different Mashups and APIs. Then, we introduce the functional semantic component to generate text embedding vectors for Mashups and APIs, enhancing their requirement semantics at multiple levels of abstraction. Finally, we fuse the output vectors to obtain the list of candidate Web APIs. Experiences are performed on real datasets, and statistical results show that FSFM outperforms other state-of-the-art models in both overall and long-tail Web API recommendations.**

***Index Terms*—Web API recommendation, Mashup Creation, Heterogeneous Network, Structural Interaction, Functional Semantic**

I. Introduction

With the development of cloud computing and micro-service architecture, an increasing number of business functions are being released as remotely accessible APIs (application programming interfaces) [1]. It has enabled the creation and deployment of Mashup to end users, which allows developers to integrate existing Web API resources and create composite Web applications [2]. For example, a mashup named *Padvark.nl* is created by composing *Google Maps*, *Google Charts*, and *Strava*. However, a vast number of Web APIs bring a sweet annoyance for developers in Mashup creation, such as addressing restrictions, limitations, and compatibility issues. ProgrammableWeb, an online API registry, has registered and published over 24,000 public APIs, each with unique features and functionalities, further adding to the complexity of API integration and management in Mashup creation.

Various Web API recommendation methods for automatic Mashup creation have been proposed, which are mainly divided into two categories: 1) Collaborative filtering (CF) based methods [3] [4]. It is common to share similar mashup experiences to enhance recommendation performance. The effectiveness of matrix factorization-based methods relies heavily on having enough historical interaction records between Mashups and APIs. 2) Content-based methods [5] [6]. These works use various techniques to explore the relationship between requirements and Web APIs. Commonly, various features, including description text, tags, topics, popularity, and quality of service (QoS) are used in topic modeling methods [5] to improve the accuracy of recommendations. In recent years, there has been a surge of research focusing on utilizing network representation [7] [8] and deep learning approaches [9] [10] [11] to enhance the accuracy of Web API recommendations. However, these methods are limited in their applicability to real-world mashup applications. 1) Textual descriptions can determine API relevance, but highly similar APIs are more likely to substitute than work together in a mashup creation. 2) Real-world APIs often lack formal semantics, leading to ambiguous definitions and boundary problems. As a result, it becomes impractical and challenging to rely solely on API usage records to match and determine their intended use. 3) Sparse invocation records overlook tail APIs, with only 1.8% of API usage in mashups covering 99% of APIs. Over-reliance on header APIs can severely limit the flexibility and scalability of the Mashup, making it challenging to adapt to changing requirements and conditions, thereby impacting request processing capabilities.

In this work, we have developed a novel method to improve the performance of the Web API recommendation method for Mashup creation, namely **F**unctional and **S**tructural **F**usion **M**odel. Proper feature representation is the cornerstone of deep network models. FSFM integrates deep neural networks to capture complex nonlinear structures and semantic information of requirements, emphasizing the significance of different feature levels to deliver compound and diverse API recommendations. Comprehensive experiments on real datasets show that FSFM is effective and outperforms the other state-of-the-art models for Web API recommendation. The main contributions of this work are summarized as follows: (1) We propose an FSFM for Mashup creation, which not only seamlessly blends text semantics and network relationships, but also adaptively updates matching weights to provide API recommendation

979-8-3503-0485-5/23/$31.00 ©2023 IEEE
DOI 10.1109/ICWS60048.2023.00025

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:07:24 UTC from IEEE Xplore. Restrictions apply.

lists. (2) We propose a MAHN by combining the affinity graph among APIs, the compatibility graph among APIs, and the bipartite graph of Mashups and APIs to exploit the latent combinatorial value of tail Web APIs, which can alleviate the problem of long-tail distribution of APIs. (3) We use a convolutional neural network as the basic component of the functional semantic component in order to enrich the semantic information of Mashup and API. (4) Based on the MAHN, we construct a multi-layer light graph convolutional network and matrix factorization in the structural interaction component to automatically capture the topological structure signals of perceptual relationships between nodes and achieve heterogeneous network embedding. (5) We conducted a comprehensive experiment on the real dataset ProgrammableWeb to evaluate. Experiment results demonstrate that FSFM improves by at least 21% in NDCG, 24% in MAP, 2% in Precision, and 15% in Recall over other state-of-the-art models, both in terms of overall and long-tail recommendations.

The rest of this work is organized as follows. Section II provides some background knowledge about the service ecosystem and proposes the construction of MAHN. Section III presents the technical details of our approach. Section IV conducts experiments and results analysis. Section V offers the related work on Web API recommendation technologies. Section VI concludes our work.

II. Heterogeneous network of Mashups and APIs

In this section, we introduce the terminology and necessary concepts for the Heterogeneous network of Mashups and APIs, including concepts related to service ecosystems, as well as basic modeling in MAHN.

A. Service Ecosystem

In large service ecosystems, each Mashup application has a composition of a set of Web APIs, whose functionality is contained in API text documents, to satisfy the application's requirements. The core concepts of the service ecosystem are defined as follows:

Definition 1. Service Ecosystem: The service ecosystem is formal as $E = (A, M)$, where $A = \{a_1, a_2, ..., a_{|A|}\}$ and $M = \{m_1, m_2, ..., m_{|M|}\}$ denote the collections of Web APIs and Mashups in the ecosystem, respectively. For every mashup $m_i \in M$ that utilizes one or more Web APIs $A_{mi} \in A$, a service usage record is generated and stored.

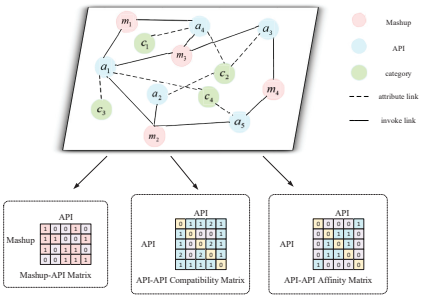

Fig. 1: Modeling of MAHN: an example.

Definition 2. Web API: A Web API is defined as $a_i = \{D_{ai}, T_{ai}, C_{ai}\}$, where D_{ai} is a text description used to introduce function information, and T_{ai} is a set of tags used to indicate its categories C_{ai}.

Definition 3. Mashup: A Mashup is defined as $m_i = \{D_{mi}, T_{mi}, C_{mi}, A_{mi}\}$, where D_{mi} is a textual description used to introduce function information, T_{mi} is a set of tags used to indicate its categories C_{mi}, and A_{mi} is a collection of API used to indicate participation in mashup applications.

Definition 4. Requirement: The mashup requirement is defined as a sequence of keywords $DW_{mi} = \{mw_1, mw_2, ..., mw_N | mw_i \in D_{mi}\}$, where N is the total number of keywords.

Definition 5. API Textual Documents: Similarly to the requirement, the API textual document is defined as a collection of keywords $DW_{ai} = \{aw_1, aw_2, ..., aw_N | aw_i \in D_{ai}\}$, where N is the total number of keywords.

B. Basic Concepts in MAHN

A plethora of Mashup applications and Web APIs, each with their own features, functionalities, and constraints, are available on the Internet. As shown in Fig. 1, an illustration of MAHN modeling comprises red, blue, and green nodes standing for mashups, APIs, and categories correspondingly, in addition to solid lines that denote invocation links and dashed lines that signify attribute links. The Mashup-API matrix identifies mashup-API interactions via invocation links, whereas the API-API compatibility matrix evaluates API collection compatibility in Mashup records. Affinity relationships are formed between APIs with similar categories, as captured by the API-API affinity matrix. With the addition of more APIs, the heterogeneous graph becomes denser. Note that compatible APIs are often used together, and APIs in the same category can be substituted in the mashup.

Definition 6. Heterogeneous Network of Mashups and APIs: A MAHN is represented by $G = (V, E)$ where V and E indicate its node set and edge set, respectively.

Definition 7. Nodes: A node in MAHN G is defined as v_m and v_a, where v_m is Mashup, v_a is Web API. Both node types have keyword sequences DW that represent their functions.

Definition 8. Edges: An edge has multiple types in MAHN G, denoted by $E = \{E_i, E_c, E_a\}$, where E_i exists when v_m invokes v_a, E_c exists when v_{a1} and v_{a2} appear in same mashup, E_i exists when v_{a1} and v_{a2} appear in the same category. Every edge E is bi-directional and assigned a weight w_{ij} to indicate the number of invocations between the nodes.

III. Functional and Structural Fusion Model

In this section, we provide a detailed explanation of the proposed approach. The overall framework, as shown in Fig. 2, consists of three main components: (1) Functional Semantic Component. It generates text embedding features for Mashup and Web API at various abstraction levels to enrich semantic information; (2) Structural Interaction Component. It efficiently learns the valuable interactions of different connections in MAHN; (3) Multi-model fusion component. It integrates different types of features to improve model performance.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:07:24 UTC from IEEE Xplore. Restrictions apply.

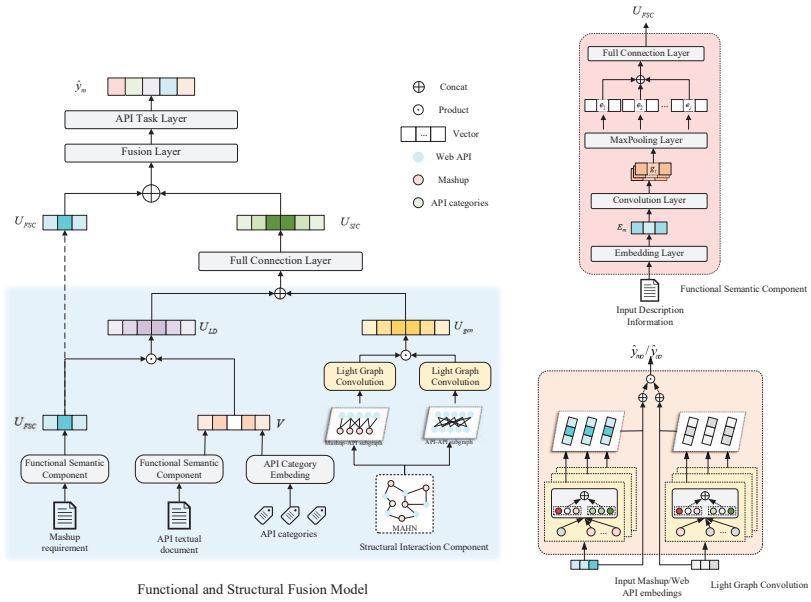

Fig. 2: Illustration of the proposed FSFM.

A. Functional Semantic Component(FSC)

The basic component of text representation in the FSC is the convolutional neural network (CNN). As shown in Fig. 2, the core of CNN is the concept of capturing local features from a sliding window consisting of multiple words, which allows FSC to automatically combine and filter features at different levels of abstraction.

In the convolution layer, a convolution kernel $G_i \in {}^{ws_j \times d}$ is initially applied by sliding it over a word window of size ws_j to perform convolution operation:

$$g_i = relu\left(G_i E_m + b_i\right) \tag{1}$$

where b_i is the bias term, E_m is the word embedding matrix for the requirement DW, and $relu$ (Rectified Linear Unit) is the activation function.

After convolution, each kernel generates a feature map g_i. These feature maps are subjected to Max-Pooling to extract significant features while eliminating redundant details.

$$e_j = [MaxPooling(g_i); ...; MaxPooling(g_I)] \tag{2}$$

where I is the number of convolution kernels.

Generally, feature extraction can be enhanced by using windows of different sizes. Hence, the extracted features are concatenated to form a comprehensive feature representation $e = [e_1; ...; e_j; ...; e_J]$ and fed into a fully connected layer.

$$U_{FSC} = sigmoid(W_0 e + b_0) \tag{3}$$

where W_0 is the weight matrix, b_0 is the bias term, sigmoid is the activation function and e_J is the results of the word window with a size of ws_j.

In order to simplify the expression, we rewrite it as $U_{FSC} = CNN\left(\theta, m\right)$. where $CNN\left(\theta, m\right)$ is the function to encode the text m and parameter $\theta = \{E_*, G_*, W_0, b_*\}$.

B. Structural Interaction Component(SIC)

The rich structural information provides help to discover hidden patterns and relationships in MAHN. Factorization techniques and graph convolution learning are the basic components of SIC to capture the topological signals of useful perceptual relationships in different dimensions.

Factorization techniques enable the projection of relational factors onto comparable regions in space, which facilitates the extraction of linear interaction features as follows:

$$U_{LD} = U_{FSC} \cdot V^T, V = \left\{\left[V_a^{FSC}; V_a^c\right] | a \in A\right\} \tag{4}$$

$$V_a^{FSC} = CNN\left(\theta_2, a\right) \tag{5}$$

where U_{FSC} is the embedding vector of mashup m, V_a^{FSC} is the functional semantic feature of Web API, and V_a^c is the mapping vector for the category of the Web API.

The linear operation between Mashups U_{FSC} and Web APIs V^T enables the model to remember the interaction patterns that are commonly occurring in historical interactions. However, it is sometimes difficult to model rare but useful latent interaction features, such as heterogeneous relationships among APIs. Thus, we introduce graph convolution learning to expand over the MAHN, which explores higher-order connectivity signals by stacking multiple light graph convolutions(LGCs).

As defined in Section II, MAHN includes various node types and multiple edge types with distinct roles and impacts on node representation. Thus, we decoupled MAHN into multiple isomorphic and bipartite subgraphs based on the edge connections between nodes. Next, we take the binary graph of Mashups and APIs as an example to illustrate how our model captures higher-order signals. For a single-layer LGC:

$$e_m^{(k+1)} = \sum_{a \in N_m} p_{ma} e_a^{(k)}; e_a^{(k+1)} = \sum_{m \in N_a} p_{ma} e_m^{(k)} \tag{6}$$

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:07:24 UTC from IEEE Xplore. Restrictions apply.

where $p_{ma} = 1/\sqrt{|N_m|\,|N_a|}$ is the Laplacian graph norm, N_m and N_a are the first-hop neighbour of mashup m and API a, respectively. $e_m^{(k)}$ and $e_a^{(k)}$ are the embeddings of mashup m and API a after propagation in the k_{th} layer, respectively.

The higher-order connectivity between mashups and APIs is crucial for encoding co-signals that are used to calculate correlation scores. Specifically, we perform k LGC operations on the embedding of 0_{th} layer to obtain the embedding representation of the k_{th} layer and aggregate the embedding representation obtained from each layer as follows:

$$e_m = \sum_{k=0}^{K} \alpha_k e_m^{(k)}; e_a = \sum_{k=0}^{K} \alpha_k e_a^{(k)} \tag{7}$$

where α_k denotes the importance of the k_{th} layer embedding in constituting the final embedding. The co-signals can be seamlessly injected into the presentation learning process by stacking multiple embedded communication layers.

Once the final embeddings of mashup m and API a are obtained, we can predict the user's preference by the inner product.

$$\hat{y}_{ma} = e_m^T e_a \tag{8}$$

After that, we also can obtain the corresponding preference predictions for the isomorphic subgraph among APIs, which consists of both the compatibility and affinity subgraphs. In order to capture this multi-type of API node dependency, Relation-aware weights β_{aa}^{sub} are used to aggregate the isomorphic subgraphs among APIs.

$$\hat{y}_{aa} = \beta_c \hat{y}_{aa}^c + \beta_a \hat{y}_{aa}^a; \beta_{aa}^{sub} = \{\beta_c, \beta_a\} \tag{9}$$

where y_{aa}^c and y_{aa}^a represent the preference prediction of the compatibility subgraph among APIs and the affinity subgraph among APIs, respectively. Notice that the set of weights β_{aa}^{sub} should not be a set of hyperparameters but should be dynamically changed according to different tasks, so we set them as trainable parameters to be learned in train.

Therefore, we generate the nonlinear interaction vector between Mashup and Web API, as follows:

$$U_{gcn} = \hat{y}_{ma} \cdot \hat{y}_{aa} \tag{10}$$

where y_{ma} and y_{aa} represent the preference prediction of the bipartite graph and the isomorphic graph, respectively.

The advantage of LGN is that it can provide global semantic and abstract details in structure interactions. With the increase of the number of layers, the receptive field of each neuron becomes larger relative to the previous layer, which is difficult to do in the linear operation. After concatenating the two feature vectors U_{LD} and U_{gcn}, we get the final output of SIC:

$$U_{SIC} = [U_{LD}; U_{gcn}] \tag{11}$$

Here, we use the concat operation to connect U_{LD} and U_{gcn} for better compatibility with the linearity of MF and the nonlinearity of LGC to model the multiple relationships.

C. Multi-model Fusion Component

In this subsection, we adopt a multi-model fusion technique that involves combining different features to improve the performance of the models. The FSC and SIC provide features from different levels and perspectives. Based on those features, we perform an aggregation operation as shown in Fig. 2.

$$\hat{y}_m = sigmoid(W_{TL} U_{MMF} + b_{TL}) \tag{12}$$

where $U_{MMF} = [U_{FSC}; U_{SIC}]$, W_{TL} and b_{TL} are the weight matrix and the bias vector in an API task layer, respectively.

We used binary cross entropy loss (BCELoss) to train the model parameters as follows:

$$\begin{aligned} Loss\left(\hat{y}_m, y_m\right) = \frac{1}{|A|} \sum_{a \in A} \left(y_m\left[a\right] \log\left(\hat{y}_m\left[a\right]\right)\right. \\ \left. + \left(1 - y_m\left[a\right]\right) \log(1 - \hat{y}_m\left[a\right])\right) \end{aligned} \tag{13}$$

$$Loss\left(\theta'\right) = \min \frac{1}{|M|} \sum_{m \in M} Loss\left(\hat{y}_m, y_m\right) + \lambda \left|\left|\theta'\right|\right|_2^2 \tag{14}$$

where $y_m\left[a\right]$ represents the target value indicating whether or not the a-th Web API exits in the mashup m, $\hat{y}_m\left[a\right]$ denotes the predicted value corresponding to $y_m\left[a\right]$, $\theta' = \theta \cup \{W_1, b_1, ..., W_z, b_z, V\}$ is the model parameters, and λ is a L_2 regularization weight.

After the model has been trained, we are able to generate a list of recommended candidate APIs for the mashup creation requirement DW_m by predicting the correlation score $\hat{y}_m$.

IV. Experiment

A. Datasets

We use a real-world dataset to conduct experiments and obtain reliable results. The dataset, which is crawled from ProgrammableWeb.com, comprises historical interaction records between 8,217 mashups and 23,518 APIs. Out of all the APIs, 1,647 Web APIs are invoked by Mashups, resulting in 17,182 interactions. More detailed information is provided in Table I. In our experiment, we used the hold-out method to randomly split the dataset into three sets: train, validation, and test set.

TABLE I: Statistic Information of the experimental dataset.

Parameters	Values
Number of APIs	23518
Number of Mashups	8217
Number of API Invocations by Mashups	17182
Number of unique APIs used in Mashups	1647
Average number of member APIs per Mashup	2.09
Average number of tags per APIs	2.447
Average number of tags per Mashups	2.271
Average number of terms in Mashup descriptions	17.019
Average number of terms in API descriptions	44.296

B. Experimetal Setting

We consider different baselines as follows to evaluate:

Traditional Collaborative Filtering(CF) methods: These methods rely on the history of API invocations. Specifically, we evaluate Collaborative Filtering (CF) [3] and Neural Collaborative Filtering (NCF) [12] in our scenario.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:07:24 UTC from IEEE Xplore. Restrictions apply.

TABLE II: Performance comparison between our method and other approaches on overall Web API recommendation.

Method	N = 5				N = 10				N = 15				N = 20			
	NDCG	MAP	Precision	Recall	NDCG	MAP	Precision	Recall	NDCG	MAP	Precision	Recall	NDCG	MAP	Precision	Recall
CF	0.2912	0.2302	0.1140	0.3234	0.3637	0.2523	0.0950	0.5337	0.3765	0.2499	0.0732	0.5961	0.3817	0.2488	0.0583	0.6277
NCF	0.4143	0.3685	0.1313	0.3796	0.4405	0.3715	0.0839	0.4777	0.4496	0.3689	0.0633	0.5270	0.4529	0.3641	0.0513	0.5544
Pop	0.3977	0.3526	0.1276	0.3630	0.4372	0.3647	0.0836	0.4740	0.4463	0.3617	0.0642	0.5277	0.4525	0.3578	0.0530	0.5703
LSTM	0.4397	0.4010	0.1328	0.3880	0.4675	0.4058	0.0861	0.4764	0.4770	0.4024	0.0659	0.5325	0.4814	0.3976	0.0528	0.5657
SPR	0.5725	0.5292	0.1791	0.5223	0.5962	0.5242	0.1136	0.6305	0.6042	0.5116	0.0888	0.7106	0.6078	0.5027	0.0735	0.7661
RWR	0.6080	0.5588	0.1924	0.5721	0.6323	0.5604	0.1153	0.6664	0.6382	0.5512	0.0862	0.7267	0.6405	0.5422	0.0696	0.7688
MTFM	0.7032	0.6746	0.2224	0.6295	0.7118	0.6685	0.1267	0.6858	0.7125	0.6621	0.0894	0.7102	0.7128	0.6583	0.0695	0.7251
FSFM	**0.9336**	**0.9209**	**0.3276**	**0.8850**	**0.9112**	**0.9112**	**0.1767**	**0.9105**	**0.9297**	**0.9065**	**0.1215**	**0.9212**	**0.9283**	**0.9034**	**0.0926**	**0.9259**

Content-based methods: These methods are usually based on the semantic features of the requirements. We consider LSTM [13] and SPR [6] as content-based methods.

Popularity-based model (Pop): Pop is the most common baseline model, which takes the category-aware invocation frequency as the API usage hotness to rank the different APIs.

Random Walks with Restart (RWR): RWR [8] is centered around the use of a knowledge graph to encode the context and requirements of the mashup with categories. It recommends relevant APIs based on the relevance score obtained.

Multi-Task Fusion Model (MTFM): MTFM is a hybrid method that both models the semantics of requirements and generates latent interactions of history records [14].

For evaluation metrics, we adopted four common metrics: Precision@N, Recall@N, MAP@N, and NDCG@N. Precision@N and Recall@N provide a single value of candidate items. In contrast, MAP@N and NDCG@N are position-sensitive metrics that apply weighted discounts to reflect the importance of item positioning. In our scenario, we have set N values to 5, 10, 15, and 20.

All models are implemented with Pytorch, and Hyper-parameters are tuned with the validation set. For our model, the intercept length of the text is set to 50, and fixed the dimension of the pre-training word vector to 200. The context window size is $\{2, 3, 4, 5\}$ with 64 convolution cores in each. The number of layers embedded in the stack propagation layer is set to 3, with the embedding size of both Mashup and Web API being 128. By default, the learning rate is set to 0.001, the training batch size is 128, Adam is used as the optimizer of the model, the regularization weight λ is set to $1e-7$, and the early stop strategy is adopted in the training phase.

C. Overall Performance Comparison

As can be seen from Table II, hybrid-based methods(e.g., MTFM, FSFM) outperform the other methods, which indicates that the rich semantic and structural information enhances model performance. Content-based methods are significantly better than the traditional CF methods, which demonstrates that textual description contains a wealth of knowledge than other types of metadata. Besides, RWR is better than SPR in most cases, especially Recall, probably due to the composition of tags being more descriptive of the developer's requirements, but it also makes it more difficult for users to manipulate. In hybrid-based methods, FSFM outperforms MTFM by 21.55% for NDCG, 24% for MAP, 10.0% - 2% for Precision, and 25% - 20% for Recall, which indicates that explicitly encoding latent relationships between APIs is better than that generated by text description. Additionally, we find that these methods are especially effective in recommending APIs when considering the top-5 and top-10 recommendations, as the average number of APIs employed in creating Mashups within the dataset is fewer than 3. As the number of recommended APIs increases, the rise in Recall diminishes gradually and the Precision value decreases.

D. Long-tail Performance Comparison

Long-tail Web API recommendations refer to those Web APIs that are not popular and used by only a few mashups. Comparing Table II and Table III, it can be seen that most methods perform differently on long-tail recommendations: Pop shows a sharp decline in each metric because it only focuses on the most popular API in each category. MTFM drops significantly because MTFM only uses text descriptions as input, which is inaccurate for features generated by tail APIs. FSFM further considers the synergistic weight of multiple relationships among different Web APIs and adaptively updates the relation-aware weights for tail APIs. Furthermore, We find a change in the relation between RWR and MTFM in Recall when N ranges from 10 to 15, which is due to RWR's query weighting strategy and random wandering that enhances the weight on tail APIs, enabling it to match more actual APIs within the top 10 to 15.

V. Related Works

Collaborative filtering(CF) methods. CF-based methods leverage shared Mashup experiences and yield reasonable results even with sparse data. Chen et al. [15] combine user-based CF with item-based CF to find appropriate API by analyzing QoS information. Botangen et al. [16] propose a geo-aware CF approach. Fletcher et al. [4] incorporate the implicit user preferences into the matrix factorization model. However, those methods often fail to prioritize requirements, which are limited by incomplete QoS information, security realities, and identity privacy. Instead, we develop a complete end-to-end model for mashup creation by combining semantic texts and network relationships.

Content-based Methods. Content-based methods rely on the matching between requirements and Web API descriptions. Xia et al. [5] propose a category-aware service clustering and distributed recommendation approach. Bao et al [7] propose a fine-grained API link prediction method for mashup creation. Shi et al. [17] propose an attention-based relational topic model. Zhao et al. [18] propose a method using feature sets and learning-to-rank. The major hurdle for content-based methods

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:07:24 UTC from IEEE Xplore. Restrictions apply.

TABLE III: Performance comparison between our method and other approaches on long-tail Web API recommendation

Method	N = 5				N = 10				N = 15				N = 20			
	NDCG	MAP	Precision	Recall	NDCG	MAP	Precision	Recall	NDCG	MAP	Precision	Recall	NDCG	MAP	Precision	Recall
CF	0.1600	0.1296	0.0641	0.0682	0.2110	0.1405	0.0672	0.1479	0.2237	0.1378	0.0580	0.1830	0.2317	0.1391	0.0481	0.2047
NCF	0.2480	0.2163	0.0931	0.1080	0.2677	0.2222	0.0618	0.1399	0.2687	0.2141	0.0509	0.1600	0.2712	0.2066	0.0420	0.1810
Pop	0.2401	0.2063	0.0977	0.1110	0.2561	0.2093	0.0595	0.1404	0.2601	0.2056	0.0458	0.1520	0.2623	0.2005	0.0412	0.1679
LSTM	0.2514	0.2210	0.0947	0.1116	0.2631	0.2221	0.0611	0.1353	0.2719	0.2159	0.0519	0.1625	0.2769	0.2101	0.0439	0.1877
SPR	0.3173	0.2884	0.1115	0.1376	0.3398	0.2818	0.0870	0.1972	0.3491	0.2704	0.0768	0.2609	0.3512	0.2606	0.0698	0.3176
RWR	0.3325	0.2945	0.1221	0.1720	0.3695	0.2997	0.0924	0.2695	0.3887	0.2964	0.0784	0.3591	0.3895	0.2798	0.0679	0.4079
MTFM	0.4212	0.3970	0.1542	0.2190	0.4255	0.3746	0.1084	0.2821	0.4349	0.3671	0.0855	0.3291	0.4344	0.3598	0.0706	0.3599
FSFM	**0.6261**	**0.5972**	**0.2779**	**0.4440**	**0.6305**	**0.5817**	**0.1748**	**0.5196**	**0.6335**	**0.5739**	**0.1288**	**0.5628**	**0.6305**	**0.5640**	**0.1023**	**0.5803**

is limited by the availability and quality of the method. Instead, we build a complete end-to-end framework to avoid the dependence on textual data quality and reduce human intervention.

Deep Learning-based Methods. The research on Web API recommendation methods based on deep neural networks has made some progress in recent years. Shi et al. [13] propose a recurrent neural network model based on tag attention. Kang et al. [9] propose a hybrid factorization machine, which captures the interaction of nonlinear features by integrating deep neural networks. Wu et al. [14] propose a neural framework based on multi-model fusion and multi-task learning to model text description information and collaborative information. Xiao et al. [10] propose a structured enhancement and attribute weakening networks to capture structural relationships and attribute information. Inspired by those efforts, we propose a heterogeneous network MAHN, trained by end-to-end supervised classification, and achieve positive results,

VI. Conclusion

In this work, a hybrid model based on deep neural networks(i.e., FSFM), is proposed for Mashup creation, which seamlessly blends semantic and network relationships. FSFM encodes various structural relationships in the MAHN to obtain embedding vectors using the structural interaction component, and generates requirement semantic vectors for Mashups and APIs through the functional semantic component. Finally, all representation vectors are combined to achieve feature fusion and thus obtain a list of candidate APIs. A series of experimental results validate that FSFM outperforms the other state-of-the-art models both in terms of overall and long-tail Web API recommendations. In the future, we will consider the dynamic factors of recommendations.

Acknowledgment

This work is supported by the National Key R&D Program of China 2022YFF0902000, the National Natural Science Foundation of China under Grant 61825205, and the Key R&D Program of Zhejiang Province 2023C01213.

References

[1] H. Zhao, S. Deng, Z. Liu, J. Yin, and S. Dustdar, "Distributed redundant placement for microservice-based applications at the edge," *IEEE Transactions on Services Computing*, vol. 15, no. 3, pp. 1732–1745, 2020.

[2] S. Roy Chowdhury, F. Daniel, and F. Casati, "Recommendation and weaving of reusable mashup model patterns for assisted development," *ACM Transactions on Internet Technology (TOIT)*, vol. 14, no. 2-3, pp. 1–23, 2014.

[3] C. Buqing, M. Tang, and X. Huang, "Cscf: A mashup service recommendation approach based on content similarity and collaborative filtering," *International Journal of Grid & Distributed Computing*, vol. 7, no. 2, 2014.

[4] K. Fletcher, "Regularizing matrix factorization with implicit user preference embeddings for web api recommendation," in *2019 IEEE international conference on services computing (SCC)*. IEEE, 2019, pp. 1–8.

[5] B. Xia, Y. Fan, W. Tan, K. Huang, J. Zhang, and C. Wu, "Category-aware api clustering and distributed recommendation for automatic mashup creation," *IEEE Transactions on Services Computing*, vol. 8, no. 5, pp. 674–687, 2014.

[6] Y. Zhong, Y. Fan, W. Tan, and J. Zhang, "Web service recommendation with reconstructed profile from mashup descriptions," *IEEE Transactions on Automation Science and Engineering*, vol. 15, no. 2, pp. 468–478, 2016.

[7] Q. Bao, J. Zhang, X. Duan, R. Ramachandran, T. J. Lee, Y. Zhang, Y. Xu, S. Lee, L. Pan, P. Gatlin *et al.*, "A fine-grained api link prediction approach supporting mashup recommendation," in *2017 IEEE International Conference on Web Services (ICWS)*. IEEE, 2017, pp. 220–228.

[8] X. Wang, H. Wu, and C.-H. Hsu, "Mashup-oriented api recommendation via random walk on knowledge graph," *IEEE Access*, vol. 7, pp. 7651–7662, 2018.

[9] G. Kang, J. Liu, B. Cao, and M. Cao, "Nafm: neural and attentional factorization machine for web api recommendation," in *2020 IEEE international conference on web services (ICWS)*. IEEE, 2020, pp. 330–337.

[10] Y. Xiao, J. Liu, G. Kang, R. Hu, B. Cao, Y. Cao, and M. Shi, "Structure reinforcing and attribute weakening network based api recommendation approach for mashup creation," in *2020 IEEE International Conference on Web Services (ICWS)*. IEEE, 2020, pp. 541–548.

[11] R. Xiong, J. Wang, N. Zhang, and Y. Ma, "Deep hybrid collaborative filtering for web service recommendation," *Expert systems with Applications*, vol. 110, pp. 191–205, 2018.

[12] X. He, L. Liao, H. Zhang, L. Nie, X. Hu, and T.-S. Chua, "Neural collaborative filtering," in *Proceedings of the 26th international conference on world wide web*, 2017, pp. 173–182.

[13] M. Shi, Y. Tang, and J. Liu, "Ta-blstm: tag attention-based bidirectional long short-term memory for service recommendation in mashup creation," in *2019 International Joint Conference on Neural Networks (IJCNN)*. IEEE, 2019, pp. 1–8.

[14] H. Wu, Y. Duan, K. Yue, and L. Zhang, "Mashup-oriented web api recommendation via multi-model fusion and multi-task learning," *IEEE Transactions on Services Computing*, vol. 15, no. 6, pp. 3330–3343, 2021.

[15] X. Chen, Z. Zheng, and M. R. Lyu, "Qos-aware web service recommendation via collaborative filtering," *Web services foundations*, pp. 563–588, 2014.

[16] K. A. Botangen, J. Yu, Q. Z. Sheng, Y. Han, and S. Yongchareon, "Geographic-aware collaborative filtering for web service recommendation," *Expert Systems with Applications*, vol. 151, p. 113347, 2020.

[17] M. Shi, Y. Zhuang, Y. Tang, M. Lin, X. Zhu, and J. Liu, "Web service network embedding based on link prediction and convolutional learning," *IEEE Transactions on Services Computing*, vol. 15, no. 6, pp. 3620–3633, 2021.

[18] H. Zhao, J. Wang, Q. Zhou, X. Wang, and H. Wu, "Web api recommendation with features ensemble and learning-to-rank," in *Big Data: 7th CCF Conference, BigData 2019, Wuhan, China, September 26–28, 2019, Proceedings 7*. Springer, 2019, pp. 406–419.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:07:24 UTC from IEEE Xplore. Restrictions apply.